

Билет 18. Lock-free и wait-free. Lock-free стек Трейбера

0. Формулировка билета

Понятия lock-free и wait-free. Lock-free стек, стек Трейбера, и его реализация.

Главная идея:

Lock-free структуры данных не используют обычный mutex для защиты всей структуры.

Вместо этого они используют атомарные операции, чаще всего CAS.

CAS — compare-and-swap / compare-exchange:

```
compare_exchange(expected, desired)
```

Смысл:

```
если текущее значение == expected:  
    заменить на desired  
    операция успешна  
иначе:  
    expected обновляется текущим значением  
    операция неуспешна
```

Lock-free алгоритмы часто имеют вид:

```
прочитать текущее состояние  
подготовить новое состояние  
попробовать CAS  
если CAS не прошёл – повторить
```

1. Почему вообще хочется lock-free

Обычный mutex прост и часто хорош.

Например:

```
std::mutex m;  
std::stack<int> s;
```

```
void push(int x) {  
    std::lock_guard<std::mutex> lock(m);  
    s.push(x);  
}
```

Проблемы mutex-a:

поток может уснуть в ядре
есть context switch
если владелец lock-a остановлен scheduler-ом, остальные ждут
возможны deadlock-и при неправильном порядке lock-ов
возможна priority inversion
на очень коротких критических секциях overhead mutex-a может быть заметным

Lock-free подход пытается избежать блокировки потока на mutex-e.

2. Важное предупреждение

Lock-free не означает:

всегда быстрее
всегда проще
нет проблем

Наоборот, lock-free код часто сложнее.

Проблемы:

сложные CAS-циклы
ABA problem
сложное управление памятью
cache ping-pong
тяжело доказывать корректность
тяжело отлаживать

На экзамене полезно сказать:

Lock-free структуры применяют не потому, что mutex всегда плох, а когда нужна определённая гарантия прогресса или когда mutex становится узким местом.

3. Progress guarantees

Есть несколько уровней гарантий прогресса.

Blocking

Обычный mutex — blocking.

Если один поток захватил mutex и завис, остальные могут ждать вечно.

```
thread A:  
  lock(m)  
  завис  
  
thread B:  
  lock(m)  
  ждёт вечно
```

То есть прогресс остальных потоков зависит от одного потока.

Obstruction-free

Obstruction-free:

Если поток выполняется в одиночку достаточно долго, он завершит операцию.

Но если есть конкуренция, гарантий мало.

Это слабая гарантия.

Lock-free

Lock-free:

Система в целом делает прогресс: среди конкурирующих потоков хотя бы один за конечное число шагов завершит операцию.

Важно:

```
lock-free не гарантирует прогресс каждому конкретному потоку
```

Один поток может долго проигрывать CAS и голодать.

Но кто-то другой будет успешно продвигаться.

Формула:

```
lock-free:
  global progress
```

Wait-free

Wait-free:

Каждый поток завершит свою операцию за ограниченное число собственных шагов.

Это сильнее lock-free.

Формула:

```
wait-free:
  per-thread progress
```

То есть нет starvation для отдельного потока.

Но wait-free алгоритмы обычно намного сложнее.

4. Сравнение гарантий

```
blocking:
  один зависший поток может остановить всех

obstruction-free:
  поток завершится, если ему не мешают

lock-free:
  хотя бы один поток всегда продвигается

wait-free:
  каждый поток завершает операцию за bounded number of steps
```

По силе:

```
wait-free > lock-free > obstruction-free > blocking
```

5. CAS

Основная операция для lock-free структур — CAS.

В C++:

```
std::atomic<Node*> head;  
  
Node* expected = head.load();  
  
bool ok = head.compare_exchange_weak(expected, desired);
```

Если успешно:

```
head был равен expected  
теперь head = desired  
CAS вернул true
```

Если неуспешно:

```
head был не равен expected  
expected обновился текущим head  
CAS вернул false
```

Есть два варианта:

```
compare_exchange_weak  
compare_exchange_strong
```

weak

`compare_exchange_weak` может завершиться неуспешно даже если значение равно `expected`.

Это называется spurious failure.

Поэтому его обычно используют в цикле.

strong

`compare_exchange_strong` не должен делать spurious failure.

Но в lock-free циклах часто используют `weak`, потому что цикл всё равно повторяется.

6. Идея lock-free стека

Обычный односвязный стек:

```
head → node1 → node2 → node3 → nullptr
```

Каждый узел:

```
struct Node {  
    T value;  
    Node* next;  
};
```

Стек хранит атомарный указатель на вершину:

```
std::atomic<Node*> head_;
```

Операции:

```
push:  
    создать новый узел  
    поставить его перед текущим head  
  
pop:  
    снять текущий head  
    сделать head = old_head->next
```

Главная проблема:

Между чтением `head` и записью нового `head` другой поток может изменить стек.

Поэтому нужен CAS.

7. Treiber stack

Treiber stack — классический lock-free стек на CAS.

Он хранит:

```
std::atomic<Node*> head_;
```

И делает `push` / `pop` через CAS-цикл.

8. Push в Treiber stack

Хотим добавить новый узел в начало.

Было:

```
head → A → B → C
```

Добавляем X.

Нужно получить:

```
head → X → A → B → C
```

Логика:

1. Создать X.
2. Прочитать old_head.
3. Сделать X->next = old_head.
4. CAS(head, old_head, X).
5. Если CAS не прошёл, повторить:
 old_head изменился
 обновить X->next
 снова CAS

Код:

```
template <typename T>
class LockFreeStack {
private:
    struct Node {
        T value;
        Node* next;

        explicit Node(T v)
            : value(std::move(v)), next(nullptr)
        {}
    };

    std::atomic<Node*> head_{nullptr};

public:
    void push(T value) {
        Node* new_node = new Node(std::move(value));

        Node* old_head = head_.load(std::memory_order_relaxed);
```

```

do {
    new_node->next = old_head;
} while (!head_.compare_exchange_weak(
    old_head,
    new_node,
    std::memory_order_release,
    std::memory_order_relaxed
));
}
};

```

Почему `new_node->next = old_head` внутри цикла?

Потому что при неуспешном CAS `old_head` обновляется текущим значением `head_`.

Например:

```

мы думали, что head = A
но другой поток сделал head = B

CAS не прошёл
old_head теперь B
нужно сделать new_node->next = B

```

9. Linearization point для push

Linearization point — момент, когда операция логически становится совершённой.

Для `push` это успешный CAS:

```
head_.compare_exchange_weak(old_head, new_node)
```

До успешного CAS новый узел ещё не в стеке.

После успешного CAS он стал новой вершиной.

То есть:

```
push считается произошедшим в момент успешного CAS
```

Это важно для доказательства корректности lock-free структур.

10. Pop в Treiber stack

Хотим снять вершину.

Было:

```
head → A → B → C
```

После `pop` должно быть:

```
head → B → C
```

Логика:

1. Прочитать `old_head`.
2. Если `old_head == nullptr`, стек пуст.
3. Прочитать `next = old_head->next`.
4. `CAS(head, old_head, next)`.
5. Если `CAS` успешен:
мы сняли `old_head`
возвращаем его `value`
6. Если `CAS` не успешен:
кто-то изменил `head`
повторяем

Код в упрощённом виде:

```
std::optional<T> pop() {  
    Node* old_head = head_.load(std::memory_order_acquire);  
  
    while (old_head != nullptr) {  
        Node* next = old_head->next;  
  
        if (head_.compare_exchange_weak(  
            old_head,  
            next,  
            std::memory_order_acquire,  
            std::memory_order_relaxed  
        )) {  
            T result = std::move(old_head->value);  
  
            // Осторожно: delete old_head здесь не всегда безопасен.  
            // Это отдельная проблема memory reclamation.  
            delete old_head;  
  
            return result;  
        }  
    }  
}
```

```

    }

    // Если CAS не прошёл, old_head обновился текущим head.
}

return std::nullopt;
}

```

Но этот код с `delete old_head` опасен в настоящем lock-free стеке.

Почему — ниже.

11. Linearization point для pop

Для успешного `pop` linearization point — успешный CAS:

```
CAS(head, old_head, next)
```

До CAS узел ещё лежал в стеке.

После CAS `head` уже указывает на следующий узел.

То есть операция `pop` логически произошла в момент успешного CAS.

Для пустого стека:

```
old_head == nullptr
```

linearization point — чтение `head`, которое увидело `nullptr`.

12. Почему Treiber stack lock-free

Если несколько потоков одновременно делают `push` или `pop`, они могут мешать друг другу.

Например, два потока одновременно делают `push`.

Оба прочитали старый `head = A`.

```

thread 1 хочет head = X
thread 2 хочет head = Y

```

Только один CAS сможет пройти.

Например:

```
thread 1 CAS успешен:
  head = X

thread 2 CAS неуспешен:
  old_head обновился
  thread 2 повторяет цикл
```

Если CAS одного потока не прошёл, это обычно означает:

Другой поток успешно изменил `head`.

То есть система в целом продвинулась.

Поэтому Treiber stack является lock-free:

```
отдельный поток может долго проигрывать CAS
но каждый проигрыш означает, что кто-то другой сделал прогресс
```

13. Почему Treiber stack обычно не wait-free

Wait-free требует, чтобы каждый поток завершил операцию за ограниченное число собственных шагов.

В Treiber stack поток может бесконечно проигрывать CAS из-за конкуренции.

Сценарий:

```
thread A:
  читает head
  готовит CAS
  другой поток меняет head
  CAS не проходит
  повторяет
  снова кто-то меняет head
  CAS не проходит
  ...
```

Глобальный прогресс есть, но конкретный поток может голодать.

Поэтому:

Treiber stack:
lock-free, но не wait-free

14. Memory order для Treiber stack

Для базового экзаменационного ответа можно не уходить слишком глубоко, но полезно понимать смысл.

В `push`:

```
head_.compare_exchange_weak(  
    old_head,  
    new_node,  
    std::memory_order_release,  
    std::memory_order_relaxed  
)
```

Почему release?

Потому что перед публикацией узла мы записали:

```
new_node->value = ...  
new_node->next = old_head
```

После успешного CAS новый узел становится видимым другим потокам через `head_`.

Нужно, чтобы другой поток, который сделает `pop`, увидел корректно и `value`, и `next`.

В `pop`:

```
head_.load(std::memory_order_acquire)
```

или успешный CAS с `acquire`.

Почему `acquire`?

Потому что поток читает узел, который был опубликован через `release`.

Связка:

```
push release  
pop acquire
```

даёт корректную видимость данных внутри узла.

Упрощённо можно сказать:

push публикует узел через release, pop читает опубликованный узел через acquire.

15. Важная проблема: memory reclamation

Наивный `pop` после успешного CAS делает:

```
delete old_head;
```

Но в lock-free структуре это может быть опасно.

Сценарий:

```
thread A:
    old_head = head.load()    // получил указатель на узел A
    next = old_head->next     // собирается прочитать

thread B:
    pop()
    успешно снял тот же узел A
    delete A

thread A:
    продолжает работать с old_head
    old_head уже освобождён
    use-after-free
```

То есть даже если узел уже логически удалён из стека, другой поток мог ранее прочитать указатель на него и ещё не закончить работу.

Поэтому настоящая реализация Treiber stack требует отдельной схемы управления памятью:

```
hazard pointers
epoch-based reclamation
reference counting
atomic_shared_ptr
garbage collection
```

Подробно это будет в билете про memory reclamation.

Для текущего билета достаточно сказать:

Treiber stack как алгоритм CAS над head простой, но безопасное удаление узлов — отдельная сложная проблема.

16. ABA problem

У Treiber stack есть ещё одна классическая проблема — ABA.

CAS проверяет только:

```
head == old_head
```

Но он не знает, что происходило с этим указателем между чтением и CAS.

Сценарий:

Изначально:

```
head = A  
A->next = B
```

thread 1:

```
old_head = A  
next = B  
приостановился
```

thread 2:

```
pop A  
pop B  
push A обратно
```

Теперь:

```
head снова A  
но стек уже другой
```

thread 1:

```
CAS(head, A, B) проходит
```

Для thread 1 кажется, что ничего не изменилось:

```
было A  
стало опять A
```

Но структура между этим изменилась.

Это и есть ABA:

$A \rightarrow B \rightarrow A$

CAS видит только одинаковое значение указателя, но не видит историю.

В стеке это может привести к повреждению структуры.

17. Способы борьбы с АВА

Основные идеи:

1. Tagged pointer

Храним не только указатель, но и счётчик версии:

$(\text{pointer}, \text{counter})$

Каждое изменение head увеличивает counter.

Тогда было:

$(A, 1)$

Потом:

$(B, 2)$

Потом снова указатель A, но версия другая:

$(A, 3)$

CAS уже не спутает:

$(A, 1) \neq (A, 3)$

2. Hazard pointers

Поток заранее объявляет:

я сейчас работаю с этим узлом

Пока узел находится в hazard pointer-е какого-то потока, другой поток не имеет права его удалять.

Это помогает и с memory reclamation, и частично с ABA, потому что узел не может быть быстро освобождён и переиспользован.

3. Epoch-based reclamation

Узлы не удаляются сразу.

Они складываются в retired list и освобождаются только когда известно, что все потоки вышли из старой эпохи и уже не могут держать старые указатели.

4. Garbage collection

В языках с GC проблема освобождения памяти проще: узел не будет уничтожен, пока на него есть ссылки.

Но ABA всё равно может существовать логически, если узел удалили и добавили обратно.

18. True lock-free структура состоит из двух частей

Для lock-free стека недостаточно написать CAS-цикл.

Нужно решить две задачи:

1. Логическая корректность структуры:
push/pop через CAS над head
2. Безопасное управление памятью:
когда можно delete удалённый узел?

Первая часть — Treiber stack.

Вторая часть — memory reclamation.

На экзамене это хороший вывод:

Сам Treiber stack короткий, но настоящая промышленная реализация сложна из-за ABA и memory reclamation.

19. Почему lock-free может плохо масштабироваться

Даже без mutex-а все потоки всё равно конкурируют за один `head`.

`head` лежит в одной cache line.

Если много потоков часто делают `push/pop`, они постоянно делают CAS по одному адресу.

Это создаёт:

```
contention
cache ping-pong
много неуспешных CAS
```

То есть lock-free не равно “без contention”.

Пример:

```
mutex:
  contention на lock

Treiber stack:
  contention на atomic head
```

Иногда lock-free быстрее, иногда нет.

20. Где Treiber stack уместен

Lock-free stack может быть полезен:

```
free-list
пул объектов
внутренние runtime-структуры
ситуации, где нужен неблокирующий прогресс
короткие операции push/pop
```

Но как общая структура данных для всего подряд он не всегда хорош.

Проблемы:

```
LIFO не всегда подходит
contention на head
```

сложная memory reclamation
ABA

21. Упрощённый полный код Treiber stack

Этот код показывает идею, но в реальности опасен без правильного memory reclamation.

```
#include <atomic>
#include <optional>
#include <utility>

template <typename T>
class TreiberStack {
private:
    struct Node {
        T value;
        Node* next;

        explicit Node(T value)
            : value(std::move(value)), next(nullptr)
        {}
    };

    std::atomic<Node*> head_{nullptr};

public:
    void push(T value) {
        Node* new_node = new Node(std::move(value));

        Node* old_head = head_.load(std::memory_order_relaxed);

        do {
            new_node->next = old_head;
        } while (!head_.compare_exchange_weak(
            old_head,
            new_node,
            std::memory_order_release,
            std::memory_order_relaxed
        ));
    }

    std::optional<T> pop() {
        Node* old_head = head_.load(std::memory_order_acquire);

        while (old_head != nullptr) {
            Node* next = old_head->next;

            if (head_.compare_exchange_weak(
```

```

        old_head,
        next,
        std::memory_order_acquire,
        std::memory_order_relaxed
    )) {
        T result = std::move(old_head->value);

        // Упрощение:
        // в настоящей lock-free структуре просто delete может быть
unsafe.
        delete old_head;

        return result;
    }

    // old_head обновился текущим head после неуспешного CAS
}

return std::nullopt;
}
};

```

Что обязательно проговорить про этот код:

```

push:
    публикует новый node через CAS head

pop:
    снимает head через CAS head

успешный CAS:
    linearization point операции

CAS failed:
    кто-то другой изменил head,
    значит система сделала прогресс

delete old_head:
    в реальной реализации опасен без memory reclamation

```

22. Что сказать коротко

Lock-free и wait-free — это гарантии прогресса. Blocking-алгоритм может остановить всех, если один поток захватил mutex и завис. Lock-free означает, что система в целом делает прогресс: хотя бы один поток за конечное число шагов завершит операцию. Но отдельный поток может голодать. Wait-free сильнее: каждый поток завершает операцию за ограниченное число собственных шагов.

Классический пример lock-free структуры — стек Трейбера. Он хранит атомарный указатель `head` на вершину односвязного списка. `push` создаёт новый узел, ставит `new_node->next = old_head` и пытается через CAS заменить `head` с `old_head` на `new_node`. Если CAS не прошёл, значит другой поток изменил `head`, поэтому операция повторяется с новым `old_head`.

`pop` читает `old_head = head`, если стек пуст — возвращает пустой результат. Иначе берёт `next = old_head->next` и CAS-ом пытается заменить `head` с `old_head` на `next`. Успешный CAS — linearization point: именно в этот момент узел считается снятым со стека. Если CAS не прошёл, значит кто-то другой изменил стек, и мы повторяем попытку.

Treiber stack является lock-free, потому что если CAS одного потока не проходит, это означает, что другой поток успешно изменил `head`, то есть система в целом продвинулась. Но он не wait-free: конкретный поток может бесконечно проигрывать CAS при высокой конкуренции.

Главные сложности Treiber stack — ABA problem и memory reclamation. ABA возникает, когда `head` был `A`, потом изменился, а потом снова стал `A`, и CAS не видит, что структура успела поменяться. Memory reclamation нужна, потому что нельзя просто `delete`-ить снятый узел: другой поток мог раньше прочитать указатель на него и ещё использовать. Для решения используют tagged pointers, hazard pointers, epochs, reference counting или `atomic_shared_ptr`.

23. Мини-проверка

1. Почему обычный mutex считается blocking-примитивом?
2. Что означает obstruction-free?
3. Что означает lock-free?
4. Что означает wait-free?
5. Чем wait-free сильнее lock-free?
6. Что делает CAS / compare_exchange?
7. Как устроен push в Treiber stack?
8. Как устроен pop в Treiber stack?
9. Что такое linearization point?
10. Почему Treiber stack lock-free?
11. Почему Treiber stack обычно не wait-free?
12. Какие две главные проблемы есть у Treiber stack?